

Day 3

Object-Oriented Programming in C++

Class and Object

Definition:

- **Class:** A user-defined data type that groups **data members** (variables) and **member functions** (methods).
- **Object:** A variable (instance) of a class.
- Even an empty class has a size of **1 byte** (for unique object identification).

Example:

```
class MyClass {  
private:  
    int num;    // Data Member    //BY default Private  
public:  
    void setNum(int); // Member Function declaration    //by default inline  
    int getNum();    // Member Function declaration  
};
```

Object Creation:

```
MyClass m1;    //object on stack  
MyClass *m2 = new MyClass;    //object on heap & pointer which pointing to that object is on stack    //to release the memory must  
use delete.  
m1.setNum(10);    // Function call using object  
  
cout << m1.getNum();
```

Empty Class

```
class MyClass {};  
MyClass m1;  
cout << sizeof(m1); // Output:
```

Array of Object

```
// Dynamic memory allocation for array of objects  
MyClass* m = new MyClass[req]; // allocates memory for 'req' objects on heap  
  
// Input and assign values to each object  
for (int i = 0; i < req; i++) {  
    cin >> num;  
    m[i].setNum(num); // calling member function for each object  
}  
  
// Deallocate memory (comparison shown)  
delete m; // deletes only the first object (incorrect for arrays)  
delete[] m; // deletes all objects in the array (correct way)  
// calls destructor for all objects automatically  
//[] : Subscript operator
```

Defining Member Functions

Member functions can be:

1. **Inside class definition** → implicitly **inline**. → **Private**.
2. **Outside class definition** using the **scope resolution operator (::)**. //Not by default inline

```
void MyClass::setNum(int k) {  
    this->num = k; // 'this' points to invoking object  
    or  
    (*this).num = k;  
}
```

Can make inline using inline keyword

```
inline void setNum(int);           //Declaration inside the class  
inline void MyClass::setNum(int k) { num = k; } //Definition outside the class
```

this Pointer

- this is a pointer pointing to the **current (invoking) object**.
- Used to distinguish between class members and local variables.
- Not available in **static member functions** (because static members are not tied to any object).

Constructors

Definition:

A **special member function** automatically called when an object is created. (Same as **JAVA**).

Rules:

- Same name as the class.
- No return type.
- Called automatically when object is created.
- Cannot be declared const, static, or virtual.
- Can be overloaded

Types of Constructors:

1. **Default / No-args` Constructor**
2. **Parameterized Constructor**
3. **Copy Constructor**

Compiler's Role in Constructors (C++):

- If **no constructor** is defined by the programmer, the compiler automatically provides:
 - **Default (no-argument) constructor**
 - **Copy constructor**

- If **any constructor** (default or parameterized) is defined by the programmer,
 - Compiler always provides **copy constructor** automatically.
 - If a **copy constructor** is also defined by the programmer,
 - **No constructor is generated automatically** by the compiler.
-

a) Default Constructor

Automatically initializes objects with default values.

```
class MyClass {  
private:  
    int num;  
public:  
    MyClass() { num = 0; cout << "Default constructor\n"; }  
};
```

b) Parameterized Constructor

Takes arguments to initialize objects.

```
MyClass(int k) { num = k; }
```

c) Copy Constructor

Copy constructor is called when an object is initialized using another object.

```
MyClass m1;    // default or no-arg constructor  
MyClass m2 = m1; // copy constructor  
// or  
MyClass m2(m1);
```

Here, m2 is initialized with the help of m1.

Copy constructor will be invoked for the m2 object.

Copy Constructor is Invoked When:

1. When one object is initialized with the help of another object.
Example: `MyClass m2 = m1;` (While creating)
2. If the object is already created and we write `m2 = m1;` — this is **assignment**, not initialization.
3. When an object is **passed by value** to a function.
In this case, the compiler creates a **temporary object** using the copy constructor.

```
MyClass m1(100);  
void disp(MyClass temp) // MyClass temp = m1  
{  
}  
disp(m1);
```

Important Points

- Copy constructor is the one which takes **class reference** as an argument.

- If we directly pass the object as an argument (not by reference), it will go into **recursive calls** — because to pass an object by value, a copy must already be made, which again calls the copy constructor.
 - While declaring the copy constructor manually, we must copy **all the data members** of the original object.
-

Example

```
MyClass m1(100);  
MyClass m2 = m1; // m2 goes implicitly, m1 goes manually
```

Initializing stack-based object from another object:

```
MyClass(const MyClass& obj) {  
    this->num = obj.num;  
}
```

Heap and Stack Based Initialization

Heap-based object initialized using stack-based object:

```
MyClass* m3 = new MyClass(m1);
```

Stack-based object initialized using heap-based object:

```
MyClass m4 = *m3;
```

When to Write Copy Constructor Explicitly

- When the class contains **pointers** or uses **Dynamic Memory Allocation (DMA)**.
-

Shallow Copy

- The compiler's default copy constructor just copies the **address** of dynamically allocated data.
 - Dangerous when pointers are used because both objects will point to the **same memory location**.
-

Deep Copy

- A user-defined copy constructor allocates **new memory** and safely copies the **contents**.
-

Why We Need to Provide the Destructor in Such Cases

- To **deallocate heap-based memory**.
 - In the case of shallow copy, if the destructor automatically gets called for the second object, the dynamically allocated memory (DMA) will be released once.
Later, when the first object's destructor runs, it will try to delete the **same memory again**, leading to undefined behavior.
Hence, a proper destructor is required to manage memory correctly.
-

d) Explicit Copy Constructor Example

```
MyClass(const MyClass& ref) {  
    name = new char[strlen(ref.name) + 1];  
    strcpy_s(name, strlen(ref.name) + 1, ref.name);  
    age = ref.age;  
}
```

When to Define Your Own Copy Constructor:

- When the class contains **pointers** or **dynamically allocated memory**.

Destructor

Definition:

- A special member function that releases resources before an object is destroyed.
- Called automatically when an object goes out of scope.
- If not provided explicitly, the compiler provides a default one.
- Same name as class, preceded by ~.
- No return type.
- Cannot be overloaded.

Flow of Execution

Destructor is called. → Members are released or destroyed.

Syntax:

```
~MyClass() {  
    cout << "Inside destructor" << endl;  
    delete[] ptr;  
}
```

Rules

- Same name as class but preceded by ~.
- No return type or parameters.
- Called automatically when:
 - Stack-based object goes out of scope.
 - Explicitly for heap-based objects using delete.
 - If we allocate dynamic memory (DMA) inside the constructor, we must release that memory inside the destructor to **avoid memory leaks**.
 - When a pointer is a member of the class and memory is allocated for it on the heap, we must define the destructor explicitly because this job **cannot be done by the compiler's destructor**.

Example:

```
MyClass* m = new MyClass();  
delete m; // Explicitly calls destructor
```

Heap vs Stack Objects

Type	Destructor Call
Stack object	Automatically called
Heap object	Must use delete explicitly

Const Keyword

Const data Member:

- Declared with the const keyword.
- Constructor & Destructor Cannot be Constant.
- Const instance member must initialize in the same line or inside the constructor.
- If we doesn't initialize the const member within the class we must need to initialized them into all the constructors.

```
class MyClass
{
private:
    const int cnt=9; // no need to initialize in the constructors
    //const int cnt; // you must initialize in the constructor
    int num=5;
public:
    MyClass(int k) // no need to initialize "cnt" here as it has been initialized above
    {
        num = k;
    }

    if you haven't initialize "cnt" above
    MyClass(int a, int b) :cnt(b)
    {
        num = a;
    }
}
```

Const data Member:

```
void disp()
{
    //Normal function can change the value of non const member but cannot const member
    //cnt++; //Error
    num++; // No Error
    cout << endl << num << "\t" << cnt << endl;
}

void myfun()const
{
    //Cannot change the value of any variable (Const or Non Const).
    //cnt++; //Error
    //num++; //Error
}

};
```

Const Object:

- It can only invoke const member function.
- Normal Function can call both (const & non const member functions).

Mutable Keyword

- When we want to **modify a non-const data member** inside a **const member function**, even though other members should remain unchanged, we use the **mutable keyword**.
- By default, a **const member function** cannot modify any data member of the class.
- To allow modification of a specific member inside a const function, that member must be declared as **mutable**.
- mutable allows that particular data member to be **modified** even when the object is **const** or the function is **const**.

Syntax:

```
class MyClass {  
    mutable int count; // mutable member  
    int value;  
public:  
    void show() const {  
        count++; // allowed because count is mutable  
        // value++; // not allowed  
    }  
};
```

Static Members

1. Static Data Member

- In old C++, while declaring any static member inside the class:
 - o Declare it **inside the class**.
 - o Define it **outside the class** (only once).

```
class MyClass {  
    static int cnt; // declaration inside the class  
};  
  
int MyClass::cnt = 30; // definition outside the class
```

Important Points

1. Static data member **belongs to the class**, not to any specific object.
2. It is **shared among all the instances** of the class.
3. It must be **defined outside the class**, otherwise the linker will throw an error.
4. All objects **share the same copy** of the static variable.
5. Changes made by one object are **reflected in all objects**.

Example:

```
class MyClass {  
    static int count;  
public:  
    void inc() { count++; }  
    void show() { cout << count << endl; }  
};  
  
int MyClass::count = 0; // definition
```

```
int main() {
    MyClass m1, m2;
    m1.inc();
    m2.inc();
    m1.show(); // Output: 2
}
```

2. Static Member Function

- Static member functions can **access only static data members** of the class.
- They **do not depend on any object**, hence they cannot access this pointer.
- They are called using the **class name** instead of the object name.

Syntax:

```
class MyClass {
    static int count;
public:
    static void show() {
        cout << count;
    }
};

int MyClass::count = 10;

int main() {
    MyClass::show(); // called using class name
}
```

3. Static const Members

1. static const members are **shared by all objects**, and only **one copy** exists for the class.
2. If a const member is also static, then it **cannot have separate copies** for each object.
3. If a static const member is **initialized inside the class**, it does **not need to be initialized** inside the constructor — because static members are defined **outside** the class.
4. static has **higher precedence** than const.

Example:

```
class MyClass {
    static const int size = 10; // initialization allowed inside the class
};
```

4. Static inline (Introduced in C++17)

1. Before C++17, static members had to be **defined outside the class**.
2. From **C++17 onwards**, using the **inline** keyword allows you to **define and initialize static members directly inside the class**.
3. This eliminates the need for a separate definition outside the class.

Example:

```
class MyClass {
    static inline int cnt = 20; // defined directly inside the class
};
```

Summary Table

Feature	Static Data Member	Static Function	Static const / inline
Belongs to	Class (shared)	Class (shared)	Class (shared)
Access type	Via class or object	Via class only	Via class
Can access non-static?	No	No	No
Defined outside class	Yes (till C++17)	No	No (if inline used)
Shared among objects	Yes	Yes	Yes

Singleton Class

- Singleton is a design pattern that ensures **only one instance** of a class exists throughout the program and provides a **global access point** to that instance.

Need:

- Used when only one object is required such as:

Thread pools, caches, loggers, configuration managers, device drivers, etc.

- Creating multiple instances may lead to **incorrect behavior, resource wastage, or inconsistent results**.

Key Features

1. Only **one instance** of the class can be created.
2. Constructor is **private** (to restrict direct creation).
3. Copy constructor is **private** (to prevent copying).
4. A **static member** holds the single instance.
5. A **static function** provides global access to that instance.

Example:

```
#include<iostream>
using namespace std;

class Single{
private:
    static Single s;    // static instance
    Single(){           // private constructor
        cout << "in default constructor\n";
    }
    Single(Single& ref){ // private copy constructor
        cout << "in copy constructor\n";
    }
public:
    void disp(){
        cout << "in disp method\n";
    }
    static Single& getSingle(){ // static function to access instance
        return s;
    }
}
```

```
};

Single Single::s; // definition of static instance

int main(){
    Single& ref = Single::getSingle(); // access single instance
    ref.disp();
}
```

Explanation

- Single::s → single static instance created only once.
 - getSingle() → returns reference to that same object.
 - Constructor and copy constructor are private → prevents direct creation or copying.
 - Hence, only **one object** of Single class exists in the entire program.
-

Summary

Property	Purpose
Private constructor	Prevent direct object creation
Private copy constructor	Prevent object duplication
Static instance	Holds the single object
Static getter function	Provides global access
Only one instance	Ensures consistent behavior
